Python Database Practicals

MongoDB Practical Questions 8–13

PyMongo connection, insert, retrieve, update, delete and JSON operations.

Before you start: Install PyMongo using `pip install pymongo` and make sure MongoDB Server is running locally. These practicals use database `university_db` and collection `students_data`.

Execution order: Run Practical 8, then 9, then 10–13 in order. Practical 9 prepares the sample documents used by the later practicals.

Practical 8: Connect Python to MongoDB

Question: Use pymongo to connect to MongoDB, handle connection errors, and select the database and students_data collection.

```
# Install first: pip install pymongo
import pymongo
from pymongo.errors import ConnectionFailure, ServerSelectionTimeoutError

try:
    client = pymongo.MongoClient(
        "mongodb://localhost:27017/",
        serverSelectionTimeoutMS=5000
    )

    # Check that MongoDB server is available
    client.admin.command("ping")

    db = client["university_db"]
    students = db["students_data"]

    print("MongoDB connected successfully!")
    print("Database:", db.name)
    print("Collection:", students.name)

except (ConnectionFailure, ServerSelectionTimeoutError) as e:
    print("MongoDB connection failed:", e)

finally:
    if "client" in locals():
        client.close()
```

Sample output:

```
MongoDB connected successfully!
Database: university_db
Collection: students_data
```

Note: Make sure MongoDB Server is running on your computer before executing this program.

Practical 9: Insert Student Documents

Question: Insert 6–8 student documents using insert_one() and insert_many(), with varied enrollments and marks including Pending.

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
db = client["university_db"]
students = db["students_data"]

# Run this practical on a fresh collection, or clear old practice data first.
students.delete_many({})
students.create_index("email", unique=True)

# Insert one document
student = {
    "name": "Aarav Patel",
    "email": "aarav@gmail.com",
    "major": "Physics",
    "year": 2,
    "enrollments": [
        {"course": "Mechanics", "marks": 82},
        {"course": "Optics", "marks": "Pending"}
    ]
}
result = students.insert_one(student)
print("One student inserted. ID:", result.inserted_id)

# Insert multiple documents
student_list = [
    {
        "name": "Diya Shah", "email": "diya@gmail.com",
        "major": "Physics", "year": 1,
        "enrollments": [{"course": "Mechanics", "marks": 91}]
    },
    {
        "name": "Kabir Mehta", "email": "kabir@gmail.com",
        "major": "Computer Science", "year": 2,
        "enrollments": [{"course": "Python", "marks": 88}]
    },
```

```
    {
        "name": "Mira Joshi", "email": "mira@gmail.com",
        "major": "Computer Science", "year": 3,
        "enrollments": [{"course": "Database", "marks": "Pending"}]
    },
    {
        "name": "Ishaan Desai", "email": "ishaan@gmail.com",
        "major": "Mathematics", "year": 1,
        "enrollments": [{"course": "Algebra", "marks": 79}]
    },
    {
        "name": "Anaya Trivedi", "email": "anaya@gmail.com",
        "major": "Mathematics", "year": 2,
        "enrollments": [{"course": "Calculus", "marks": 86}]
    },
    {
        "name": "Reyansh Rao", "email": "reyansh@gmail.com",
        "major": "Chemistry", "year": 1,
        "enrollments": [{"course": "Organic Chemistry", "marks": 75}]
    }
]

result = students.insert_many(student_list)
print("Multiple students inserted:", len(result.inserted_ids))
print("Total students:", students.count_documents({}))

client.close()
```

Sample output:

```
One student inserted. ID: <generated ObjectId>
Multiple students inserted: 6
Total students: 7
```

Note: The ObjectId shown in the output is generated automatically and will be different on your computer.

Practical 10: Retrieve Student Documents

Question: Perform basic, filtered, sorted, paginated and aggregation-based retrieval operations on students_data.

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
students = client["university_db"]["students_data"]

# 1. Display all students
print("\nAll students:")
for doc in students.find():
    print(doc["name"], "-", doc["major"])

# 2. Filter Physics students
print("\nPhysics students:")
for doc in students.find({"major": "Physics"}):
    print(doc["name"], doc["email"])

# 3. Sort by name (ascending)
print("\nStudents sorted by name:")
for doc in students.find().sort("name", 1):
    print(doc["name"])

# 4. Pagination: show 2 records on page 2
print("\nPage 2:")
for doc in students.find().sort("name", 1).skip(2).limit(2):
    print(doc["name"])

# 5. Aggregation: count students in each major
pipeline = [
    {"$group": {
        "_id": "$major",
        "total_students": {"$sum": 1}
    }},
    {"$sort": {"_id": 1}}
]

print("\nStudent count by major:")
for result in students.aggregate(pipeline):
    print(result["_id"], ":", result["total_students"])

client.close()
```

Sample output:

```
All students:
```

```
Aarav Patel - Physics
... (remaining records)

Physics students:
Aarav Patel aarav@gmail.com
Diya Shah diya@gmail.com

Page 2:
<names depend on sorted records>

Student count by major:
Computer Science : 2
Chemistry : 1
Mathematics : 2
Physics : 2
```

Note: Output order for sorted and paginated records depends on the stored documents.

Practical 11: Update Student Documents

Question: Modify student details, add/update/remove enrollments, and perform an insert/upsert operation.

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
students = client["university_db"]["students_data"]

# 1. Modify student details
result = students.update_one(
    {"email": "aarav@gmail.com"},
    {"$set": {"year": 3, "phone": "9876543210"}}
)
print("Student details modified:", result.modified_count)

# 2. Add a new enrollment
students.update_one(
    {"email": "aarav@gmail.com"},
    {"$push": {"enrollments": {
        "course": "Quantum Physics", "marks": "Pending"
    }}}
)
print("Enrollment added.")

# 3. Update an existing enrollment
students.update_one(
    {"email": "aarav@gmail.com",
     "enrollments.course": "Mechanics"},
    {"$set": {"enrollments.$.marks": 90}}
)
print("Enrollment updated.")

# 4. Remove an enrollment
students.update_one(
    {"email": "aarav@gmail.com"},
    {"$pull": {"enrollments": {"course": "Optics"}}}
)
print("Enrollment removed.")

# 5. Upsert: update if found; insert if not found
result = students.update_one(
    {"email": "newstudent@gmail.com"},
    {"$set": {
        "name": "New Student",
        "major": "Physics",
        "year": 1,
        "enrollments": []
    }},
    upsert=True
)

print("Upsert completed. Inserted:", result.upserted_id is not None)

client.close()
```

Sample output:

```
Student details modified: 1
Enrollment added.
Enrollment updated.
Enrollment removed.
Upsert completed. Inserted: True
```

Note: An upsert inserts a new document when no document matches the filter.

Practical 12: Delete Student Documents

Question: Delete a specific student by email and remove multiple students belonging to a specified major; print deleted document counts.

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
students = client["university_db"]["students_data"]

# Delete one student using email
result_one = students.delete_one({"email": "reyansh@gmail.com"})
print("One student deleted:", result_one.deleted_count)

# Delete all students of a specified major
result_many = students.delete_many({"major": "Mathematics"})
print("Mathematics students deleted:", result_many.deleted_count)

client.close()
```

Sample output:

```
One student deleted: 1
Mathematics students deleted: 2
```

Note: Run Practical 9 first so the sample records exist. Re-running a delete may return 0 if records are already deleted.

Practical 13: Export Physics Students to JSON Lines and Import JSON

Question: Export Physics students to JSON Lines and import 2–3 new student documents from JSON while handling duplicate/insertion errors.

```
import json
from pymongo import MongoClient
from pymongo.errors import BulkWriteError

client = MongoClient("mongodb://localhost:27017/")
students = client["university_db"]["students_data"]

# Create a unique index so duplicate email addresses are rejected
students.create_index("email", unique=True)

# 1. Export Physics students to JSON Lines (.jsonl)
physics_students = students.find({"major": "Physics"})

with open("physics_students.jsonl", "w", encoding="utf-8") as file:
    for student in physics_students:
        file.write(json.dumps(student, default=str) + "\n")

print("Physics students exported to JSON Lines.")

# 2. Prepare a JSON file containing 3 documents.
# One email below already exists, to demonstrate duplicate handling.
new_docs = [
    {
        "name": "Nisha Parmar",
        "email": "nisha@gmail.com",
        "major": "Biology",
        "year": 1,
        "enrollments": [{"course": "Botany", "marks": 84}]
    },
    {
        "name": "Om Vyas",
        "email": "om@gmail.com",
        "major": "Physics",
        "year": 1,
        "enrollments": [{"course": "Waves", "marks": "Pending"}]
    },
    {
        "name": "Duplicate Example",
        "email": "aarav@gmail.com",
        "major": "Physics",
        "year": 2,
        "enrollments": []
    }
]
```

```
with open("new_students.json", "w", encoding="utf-8") as file:
    json.dump(new_docs, file, indent=4)

# 3. Read JSON file and insert documents
with open("new_students.json", "r", encoding="utf-8") as file:
    imported_docs = json.load(file)

try:
    result = students.insert_many(imported_docs, ordered=False)
    print("Documents inserted:", len(result.inserted_ids))

except BulkWriteError as error:
    inserted_count = error.details.get("nInserted", 0)
    print("Documents inserted:", inserted_count)
    print("Some documents could not be inserted (for example, duplicate email).")

client.close()
```

Sample output:

```
Physics students exported to JSON Lines.
Documents inserted: 2
Some documents could not be inserted (for example, duplicate email).
```

Note: Because the example includes an existing email, the duplicate document is rejected by the unique index. The other valid documents can still be inserted.